

Semantic Layers, OSI, and QueryGraph AI Navigator in Apache Polaris

A comprehensive review with reference implementation

2026-05-23

Contents

The Two Semantic Layers, Open Semantic Interchange, and a Path for QueryGraph.ai AI Navigator into Apache Polaris	2
1. Two meanings of “semantic layer”	2
1.1 The physical / catalog semantic layer	2
1.2 The business / metric semantic layer	3
1.3 Why this distinction matters for the OSI/Polaris discussion	3
2. How each system defines its semantic layer	3
2.1 Apache Iceberg — schema as physical contract	3
2.2 Unity Catalog — securables, lineage, AI assets	3
2.3 Apache Gravitino — federated metadata mesh	4
2.4 OpenLineage — lineage as a typed event stream	4
2.5 QueryGraph.ai — semantic layer as a portable, signed bundle	4
2.6 Open Data Spaces (IPA Japan) — semantics across organizational and national boundaries	5
3. Open Semantic Interchange (OSI) — what the v0.2 spec actually says	5
3.1 Top-level object graph	6
3.2 What’s <i>not</i> in OSI	6
4. Apache Polaris and polaris#4522	7
4.1 The shape of “OSI in Polaris”	7
5. Proposal: QueryGraph AI Navigator inside Apache Polaris, aligned with OSI and ODS ...	8
5.1 Thesis	8
5.2 Architecture	8
5.3 Concrete mapping: OSI → Navigator bundle	9
5.4 ODS alignment	9
5.5 Why this is the <i>right</i> alignment with polaris#4522	10
5.6 Interaction with OpenLineage	11
5.7 Migration path	11
6. Comparison matrix	12
7. What’s in this repository	14
Sources	14

The Two Semantic Layers, Open Semantic Interchange, and a Path for QueryGraph.ai AI Navigator into Apache Polaris

A comprehensive review of what *semantic layer* means today across Apache Iceberg, Unity Catalog, Apache Gravitino, OpenLineage, QueryGraph.ai, and the IPA Open Data Spaces (ODS) initiative; how the Open Semantic Interchange (OSI) specification — now proposed for Apache Polaris in [polaris#4522](#) — fits into that landscape; and a concrete architectural proposal for landing the QueryGraph **AI Navigator** inside Apache Polaris in a way that aligns with OSI and ODS.

1. Two meanings of “semantic layer”

The phrase is overloaded. In any conversation about a “semantic layer,” you must first ask *whose semantics* — the engine’s or the business’s.

1.1 The physical / catalog semantic layer

This is the layer that gives data engines a *common, well-typed, evolvable view of the bytes on disk*. It tells Spark, Trino, Flink, DuckDB, and Snowflake what the columns are, what the partitions are, where the files are, what the snapshots are, who can read them, and what changed when.

System	What it calls its “semantics”	Unit of meaning
Apache Iceberg	Table format spec: schema, partition spec, sort order, snapshots, manifests	A table with a versioned schema and column-level lineage of writes
Unity Catalog (Databricks/UC OSS)	Three-level namespace <code>catalog.schema.table</code> with table/volume/function/model/ML-feature objects, lineage, tags, access controls	A securable with type-system semantics and governance
Apache Gravitino	Metalake → Catalog → Schema → Table/Fileset/Topic/Model; “unified Data + AI asset management” via federated connectors	A metadata entity projected through a connector
OpenLineage	Run, Job, Dataset, RunEvent — each carrying typed Facets (schema facet, datasource facet, data-quality facet, column-lineage facet, ...)	A dataset version plus the job run that produced it
Apache Polaris	REST catalog over Iceberg tables + namespaces with principal/role/grant policies	Same as Iceberg, plus principals and role-grant policies

The physical/catalog layer answers: “What table is this? What columns? What snapshot? Who can read it? Where did its rows come from?” It does **not** answer “What does this column mean in our business?”

1.2 The business / metric semantic layer

This is what dbt MetricFlow, Cube, LookML, AtScale, Honeydew, GoodData MAQL, Tableau’s logical model, and now **OSI** are about. It gives BI tools and LLM agents a *common, dialect-agnostic vocabulary of metrics, dimensions, relationships, synonyms, and grain*.

System	Unit of meaning
dbt MetricFlow	metric, entity, dimension, measure, time spine
Cube	cube, measure, dimension, join, segment
OSI	semantic_model → datasets → fields + cross-model metrics + relationships, all carrying ai_context
QueryGraph.ai	A four-layer bundle: Croissant dataset + CDIF discovery profiles + DID identity + ODRL policy

The business layer answers: “What is monthly_recurring_revenue? In what dialect? What does an LLM agent need to know about it? Who is allowed to use it for what?”

1.3 Why this distinction matters for the OSI/Polaris discussion

apache/polaris#4522 is, in plain language, a proposal to add the **business semantic layer** as a first-class object inside what has so far been a **catalog/physical semantic layer**. The whole interesting question is: *what should that look like such that it composes cleanly with Iceberg, with engines, with lineage tooling, and with cross-organizational AI agents?* That’s where QueryGraph and ODS become relevant.

2. How each system defines its semantic layer

2.1 Apache Iceberg — schema as physical contract

Iceberg’s “semantics” live in the **table specification**: a strongly-typed, ID-stable schema with safe evolution rules, a partition spec, a sort order, and an append-only history of snapshots. Every reader sees the same logical table because Iceberg manifests are the source of truth, not engine-local DDL. This is **bytes-precise** semantics, not business semantics. Polaris is the Apache-governed REST catalog implementation of Iceberg’s catalog API.

2.2 Unity Catalog — securables, lineage, AI assets

UC organizes everything as a **securable** under catalog.schema.object with built-in governance: grants, row/column masks, tags, attribute-based access. Beyond tables it models **volumes** (files),

functions, registered models, and feature tables, plus first-class lineage and a “Genie/AI” surface that lets natural-language queries resolve against catalog metadata. UC overlaps the business layer mostly through *tags* and *AI-context comments*, not through formal metric definitions.

2.3 Apache Gravitino — federated metadata mesh

Gravitino’s model is **Metalake** → **Catalog** → **Schema** → (**Table** | **Fileset** | **Topic** | **Model** | **Function**). It plugs into many backends through connectors (Iceberg, Hive, MySQL, Kafka, Hugging Face, file systems) and exposes a uniform REST API. It is positioned as a unified Data+AI metadata layer — closer to a *catalog of catalogs* than a metric semantic layer. The “semantics” it adds are *consistency of identifier* (one canonical name across engines) and *unification of metadata events*.

2.4 OpenLineage — lineage as a typed event stream

OpenLineage describes the same world as a stream of **RunEvent** messages. The core nouns are:

- **Job** — a process that consumes/produces datasets (in a namespace).
- **Run** — one execution of a Job, UUID-keyed.
- **Dataset** — a logical input/output identified by namespace + name.
- **Facets** — typed extensions: SchemaDatasetFacet, DatasourceDatasetFacet, ColumnLineageDatasetFacet, DataQualityMetricsInputDatasetFacet, OwnershipJobFacet, SqlJobFacet, and many more, including community facets like LifecycleStateChangeDatasetFacet.

OpenLineage is the **verbs** of the data plane (what was produced, by whom, from what, with what schema) where Iceberg/UC/Gravitino/Polaris are the **nouns** (what exists). It carries some semantic information (schemas, column lineage, ownership) but stops short of business metrics.

2.5 QueryGraph.ai — semantic layer as a portable, signed bundle

QueryGraph’s reference implementations `qg-python` and `qg-rust` are explicit: the **AI Navigator** produces a **four-layer semantic bundle** per dataset, packaged as a single JSON-LD document:

1. **Croissant** (MLCommons JSON-LD) — the semantic dataset model: **Dataset** → **FileObjects** + **RecordSets** with typed, semantically-typed **Fields** (`sameAs` → `schema.org` / W3C vocabularies). This is the *what the data means*.
2. **CDIF** (CODATA Cross-Domain Interoperability Framework) — a set of FAIR profiles (discovery, data-access, controlled-vocabularies, data-integration, universals) layered as `dcat:Dataset` with controlled-vocabulary pointers. This is the *how to find and align it*.
3. **DID** — deterministic `did:oyd:z<multibase>` document for the dataset/agent, plus `service_endpoint`. This is *who vouches for it*.
4. **ODRL** — `odrl:Policy` with permissions, prohibitions, assignees, and constraints (e.g. *index allowed for local AI Navigator, derive prohibited without separate agreement*). This is *what may be done with it*.

The output is a single signed JSON-LD object — `querygraph:AINavigatorSemanticBundle` — that an agent can fetch, verify, and use to ground an answer. That bundle is the unit of “semantic layer” in QueryGraph’s world.

2.6 Open Data Spaces (IPA Japan) — semantics across organizational and national boundaries

ODS is an architectural initiative from Japan’s IPA that extends data mesh ideas across organizations and jurisdictions. It identifies **three governance pillars** that any cross-organization data sharing must solve:

Pillar	Question it answers
DAD — Data Addressability & Discoverability	<i>Where is the data, how is it addressed, how is it discovered?</i>
OSI — Ontology & Semantic Interoperability	<i>Does my “customer” mean what your “customer” means?</i>
IUC — Identity & Usage Control	<i>Who are you, and what are you allowed to do with it?</i>

Its **Double Product Quanta Model (DPQM)** stacks two layers:

- An **upper ontology layer** under the *Open World Assumption* — flexible, evolving, LLM-friendly meaning.
- A **lower data product layer** under the *Closed World Assumption* — strictly validated, contract-bound, machine-enforceable.

Note the naming collision: **ODS’s OSI** ≠ **Polaris’s OSI**. ODS’s “OSI” is the *ontology pillar* (W3C-style RDF/OWL/SKOS world), Polaris’s “OSI” is the *Open Semantic Interchange* metric spec. They are complementary, not competing — see §5.

The fit with QueryGraph is exact and was, on the evidence of qq-python, intentional:

ODS pillar	QueryGraph layer
DAD — discoverability	CDIF discovery/access profiles + landing pages
OSI — ontology / semantic interop	Croissant JSON-LD with schema.org sameAs and controlled vocabularies
IUC — identity & usage control	DID (did:oyd) + ODRL policy

A QueryGraph bundle *is* an ODS-shaped data product wrapper.

3. Open Semantic Interchange (OSI) — what the v0.2 spec actually says

Announced by Snowflake, Salesforce, dbt Labs, and BlackRock on **2025-09-23**, with the v1.0 spec drop on **2026-01-27** and active development moving into **v0.2.0.dev0** (the in-tree DRAFT), OSI is now a working group with AWS, Atlan, Colibra, Cube, DataHub, Honeydew, Informatica, Mistral AI, RelationalAI, Starburst, ThoughtSpot, and others. The repository is open-semantic-interchange/OSI, Apache-2.0 for code and CC-BY for the spec.

3.1 Top-level object graph

```
semantic_model
├── name                (str, required)
├── description          (str)
├── ai_context           (str | { instructions, synonyms, examples, ... })
├── datasets[]          (required)
│   ├── name
│   └── source           (e.g. "sales.public.orders" – points at the *physical*
catalog)
│   ├── primary_key[] / unique_keys[][]
│   ├── description / ai_context
│   ├── fields[]
│   │   ├── name
│   │   └── expression.dialects[].{ dialect: ANSI_SQL|SNOWFLAKE|MDX|TABLEAU|
DATABRICKS|MAQL, expression }
│   ├── dimension.is_time
│   ├── label / description / ai_context
│   └── custom_extensions[]
├── custom_extensions[]
├── relationships[]
│   ├── name, from, to, from_columns[], to_columns[]
├── metrics[]
│   ├── name
│   ├── expression.dialects[]
│   ├── description / ai_context
└── custom_extensions[]
```

Three load-bearing design choices:

1. **source is a free-form catalog pointer.** OSI does not redefine physical tables. It points at them. The implementer chooses whether `sales.public.orders` resolves through Iceberg, Unity, Gravitino, or anything else. This is what makes OSI a *layer above* catalogs rather than a competitor to them.
2. **Multi-dialect `expression.dialects[]`.** Each metric or computed field carries N parallel expressions, one per dialect. The same `monthly_recurring_revenue` can be `ANSI_SQL`, `SNOWFLAKE`, `DATABRICKS`, `MDX`, `TABLEAU`, and `GoodData MAQL` simultaneously.
3. **First-class `ai_context`.** Every object can carry instructions, synonyms, and examples. This is the LLM-grounding surface and the reason OSI is being pitched as the “data readiness for AI” standard.

3.2 What’s *not* in OSI

- **No identity / signing.** A model is a YAML document, not a verifiable credential.
- **No usage control.** There is no notion of permissions, prohibitions, or assignees. OSI delegates that to the surrounding catalog (e.g. Polaris grants).
- **No discoverability profiles.** No DCAT, no landing pages, no cross-domain alignment vocabulary. The spec assumes a single organization can already find its own data.
- **No row-level lineage.** OpenLineage owns that.

These are the gaps that QueryGraph and ODS exist to fill.

4. Apache Polaris and polaris#4522

Polaris is the Apache-incubating REST catalog implementation of the Iceberg catalog API, plus principals/roles/grants. The proposal in polaris#4522 is to host **OSI semantic models as first-class objects in Polaris**, with three named use cases:

1. **AI-agent grounding** — an LLM agent loads OSI models from Polaris, reads `ai_context` for descriptions and synonyms, walks `dataset.source` to find the underlying Iceberg tables, and constructs a correct query.
2. **BI catalog browsing** — a BI tool lists all semantic models under a namespace via the Polaris API and populates its metric grid.
3. **Multi-engine metric consistency** — `monthly_recurring_revenue` defined once in Polaris; Spark/Trino/Snowflake each pick their dialect from `expression.dialects[]` and produce identical numbers.

The issue is filed and open as of 2026-05; it has no concrete design yet. That is the gap this report addresses.

4.1 The shape of “OSI in Polaris”

A minimal viable extension is straightforward and matches the existing Polaris object model:

```
Polaris
├─ Catalog
│   └─ Namespace
│       ├── Table          (existing – Iceberg)
│       ├── View           (existing – Iceberg view)
│       └─ SemanticModel (NEW – OSI document)
```

Storage: each `SemanticModel` is a versioned blob (the OSI YAML/JSON) plus a small structured index (name, version, dataset sources, metric names, `ai_context` summary) for catalog browsing. Permissions reuse the existing Polaris grant system (`USE_SEMANTIC_MODEL`, `MANAGE_SEMANTIC_MODEL`).

REST surface:

```
GET    /v1/{catalog}/namespaces/{ns}/semantic-models
POST   /v1/{catalog}/namespaces/{ns}/semantic-models
GET    /v1/{catalog}/namespaces/{ns}/semantic-models/{name}
PUT    /v1/{catalog}/namespaces/{ns}/semantic-models/{name}
DELETE /v1/{catalog}/namespaces/{ns}/semantic-models/{name}

GET    /v1/{catalog}/namespaces/{ns}/semantic-models/{name}/metrics
GET    /v1/{catalog}/namespaces/{ns}/semantic-models/{name}/metrics/{metric}?
dialect=SNOWFLAKE
```

Resolution: every `dataset.source` MUST be resolvable as either a fully qualified Polaris `namespace.table` identifier *or* an external URI. Polaris stays the system of record for what tables exist; OSI stays the system of record for what the metrics mean over those tables.

This much would satisfy the three use cases in the issue. It would *not* yet make Polaris a useful broker between organizations or AI agents, which is what §5 addresses.

5. Proposal: QueryGraph AI Navigator inside Apache Polaris, aligned with OSI and ODS

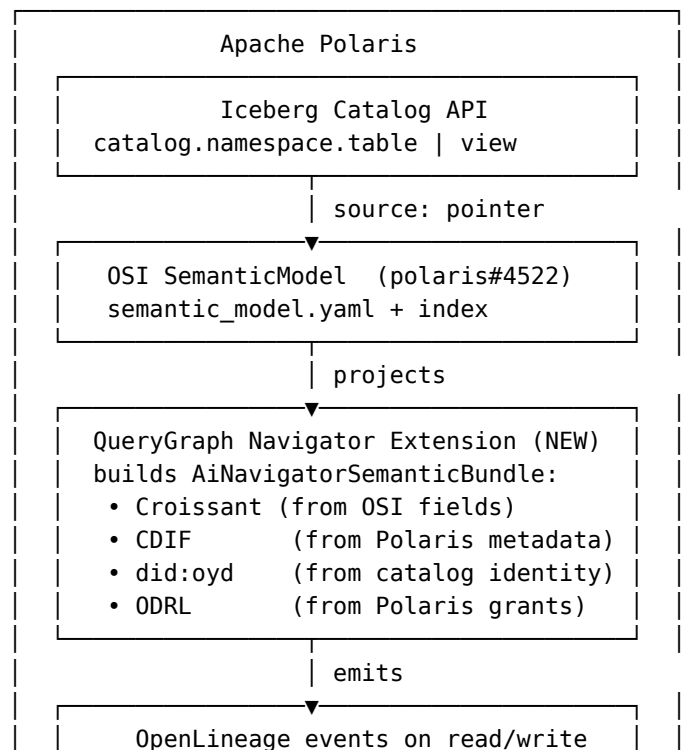
5.1 Thesis

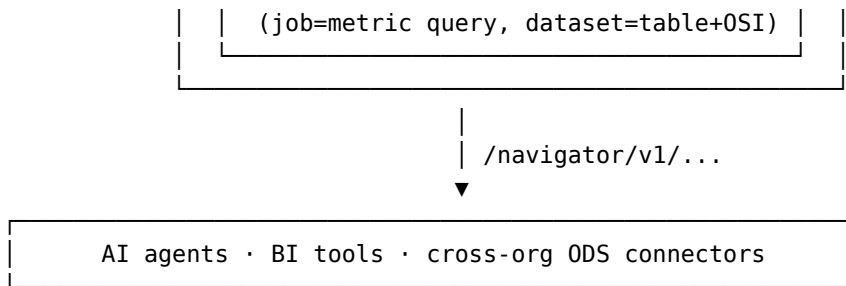
The OSI specification is the right **upper-ontology layer** for analytics inside one organization. The QueryGraph AI Navigator bundle is the right **cross-organization, agent-ready envelope** around it. Polaris should host both, and expose them at the same REST surface:

- **OSI** answers *what does this metric mean and how do I compute it?*
- **Navigator bundle** answers *who vouches for this model, where is it published, what may an AI agent do with it?*

The two are not redundant. OSI deliberately omits identity, usage control, and discoverability (see §3.2). QueryGraph’s four-layer bundle fills exactly those gaps, and does so with the same vocabularies (Croissant, DCAT, DID, ODRL) that ODS endorses.

5.2 Architecture





5.3 Concrete mapping: OSI → Navigator bundle

The transformation is deterministic and lossless in the OSI → bundle direction. Every field maps:

OSI field	Navigator bundle field
<code>semantic_model.name</code>	<code>croissant.name</code> , <code>cdif.@id</code> , DID seed
<code>semantic_model.description</code>	<code>croissant.description</code> , <code>cdif.dct:description</code>
<code>semantic_model.ai_context.instructions</code>	<code>bundle.querygraph:aiContext.instructions</code>
<code>semantic_model.ai_context.synonyms[]</code>	<code>croissant.keywords[]</code> <code>u cdif:controlledVocabulary[]</code>
<code>datasets[].source</code>	resolves to Polaris namespace.table → <code>croissant.FileObject.contentUrl</code> (Iceberg metadata URI)
<code>datasets[].fields[].name</code>	<code>croissant.RecordSet.Field.name</code>
<code>datasets[].fields[].expression.dialects[]</code>	retained verbatim in <code>bundle.osi.semantic_model</code>
<code>datasets[].fields[].ai_context.synonyms[]</code>	<code>croissant.Field.alternateName</code>
<code>relationships[]</code>	<code>croissant.RecordSet.references[]</code>
<code>metrics[]</code>	retained verbatim in <code>bundle.osi.metrics</code> ; surface name + ai_context as <code>bundle.querygraph:metricCatalog[]</code>
Polaris grants on the model	<code>odrl:Policy permissions/prohibitions</code>
Polaris principal identity	<code>did:oyd:z... document</code>

5.4 ODS alignment

The Navigator extension makes Polaris an ODS-conformant data-space node:

ODS pillar	Polaris-Navigator binding
DAD	Polaris REST /semantic-models listing + CDIF discovery profile on every bundle
OSI (ontology)	OSI semantic model + Croissant JSON-LD with controlled-vocabulary sameAs
IUC	Polaris principals → DID; Polaris grants → ODRL permission/prohibition

The ODS Double Product Quanta Model emerges naturally:

- **Upper ontology layer (OWA)** – the OSI document plus its `ai_context`, evolving, LLM-driven.
- **Lower data product layer (CWA)** – the Iceberg table referenced by source, schema-enforced, snapshot-stable. Navigator’s Croissant layer is the contract between them.

5.5 Why this is the *right* alignment with **polaris#4522**

The issue lists three use cases. The Navigator extension upgrades each:

Issue use case	Vanilla OSI-in-Polaris	1. Navigator extension
AI-agent grounding	Agent reads YAML directly from Polaris	Agent reads a signed JSON-LD bundle with usage policy and synonyms baked in; can prove provenance
BI catalog browsing	BI lists name + <code>ai_context</code>	BI lists same, <i>plus</i> knows what it’s allowed to do (ODRL) and can fetch a stable JSON-LD format already supported by Atlan/DataHub/Collibra
Multi-engine consistency	Engines pick a dialect from <code>expression.dialects[]</code>	Same; engines additionally emit OpenLineage RunEvents referencing both the table and the OSI metric, giving a <i>complete causal graph</i> over physical and semantic layers

And it adds one use case the issue does not yet enumerate: **cross-organization metric sharing**. Because a Navigator bundle is self-contained, DID-anchored, and policy-bound, one Polaris instance can publish `monthly_recurring_revenue` and another organization’s agent can consume it under ODRL terms without a federated catalog.

5.6 Interaction with OpenLineage

Every query that resolves an OSI metric through Polaris emits an OpenLineage RunEvent whose:

- `job.namespace` = the Polaris catalog
- `job.name` = the OSI metric name (e.g. `ecommerce.total_revenue`)
- `inputs[]` = the Iceberg datasets referenced by `dataset.source`
- `outputs[]` = the materialized result (or transient cache entry)
- a new `OSIMetricFacet` carries the resolved dialect and expression actually executed

This turns the lineage graph into a **two-layer graph**: physical edges (table → table) and semantic edges (metric → metric, metric → table). That graph is the canonical artifact for impact analysis when a metric definition changes.

5.7 Migration path

1. **Phase 1 — OSI as data**: Polaris stores OSI YAML blobs and exposes GET/PUT/DELETE on `/semantic-models`. No interpretation. Satisfies the bare `polaris#4522` ask.
2. **Phase 2 — Navigator projection**: A Polaris extension (in-process Java service or sidecar) projects each OSI model + the surrounding Polaris namespace into a Navigator bundle on demand. New endpoint `/semantic-models/{name}/navigator-bundle`.
3. **Phase 3 — OL emitter**: Engines reading metrics via Polaris emit `OSIMetricFacet`. Polaris ships an OL agent SDK.
4. **Phase 4 — ODS connector**: A read-only ODS data-space adapter publishes Navigator bundles into an external data space; a dual adapter consumes external bundles as virtual `SemanticModel` objects in a `__federated` namespace.

Each phase is independently shippable and useful. Nothing in phase 1 forecloses phases 2–4.

6. Comparison matrix

Aspect	Iceberg	Unity Catalog	Gravitino	Open-Lineage	OSI (Polaris)	Query-Graph Navigator	ODS
Layer	Physical	Catalog	Federated metadata	Lineage events	Metric/business	Bundle envelope	Cross-org architecture
Primary object	Table	Securable	Metalake entity	Run-Event	semantic_model	AiNavigatorSemanticBundle	Data space + product
Carries metrics?	×	partial (tags)	×	×	✓	✓ (wraps OSI)	n/a
Carries identity?	×	principals	principals	×	×	✓ DID	✓
Carries usage policy?	×	grants	grants	×	×	✓ ODRL	✓
Carries discovery profile?	×	partial	partial	×	×	✓ CDIF	✓ DAD pillar
Carries lineage?	partial (snapshots)	✓	partial	✓	×	×	n/a
AI-context fields?	×	comments	comments	×	✓	✓	OSI pillar (ontology)
Multi-dialect expressions?	×	×	×	×	✓	✓ (via OSI)	n/a
Cross-org first-class?	×	×	×	×	×	✓	✓
License	Apache 2.0	Apache 2.0 (OSS UC)	Apache 2.0	Apache 2.0	Apache 2.0 + CC-BY	Apache 2.0	open

7. What's in this repository

```
~/src/querygraph/semantic/claude/
├ semantic-layer-report.md           (this file)
├ semantic-layer-report.pdf         typeset version (pandoc + typst)
├ README.md                        short overview + nav
├ ARCHITECTURE.md                  the Polaris+Navigator design in detail
├ docs/
│   ├── two_semantic_layers.md      the §1 thesis as a standalone note
│   └ comparison_matrix.md          §6 standalone
├ examples/
│   ├── 01_osi_model_ecommerce.yaml OSI v0.2 model – e-commerce, hand-written
to compile against osi-schema.json
│   ├── 02_onlineage_event.json     OL RunEvent emitting an OSIMetricFacet
│   ├── 03_iceberg_table_metadata.json the Iceberg table the OSI model points at
│   ├── 04_gravitino_table.json     the same table as Gravitino sees it
│   ├── 05_unity_catalog_table.json the same table as UC sees it
│   ├── 06_querygraph_navigator_bundle.json four-layer bundle for the OSI model
│   ├── 07_polaris_semantic_model.json the proposed Polaris REST resource
│   └ 08_ods_data_product_manifest.yaml ODS DPQM-style manifest wrapping
the bundle
├ src/
│   ├── osi_loader.py               load + validate OSI v0.2 YAML
│   ├── navigator_from_osi.py       OSI → Navigator bundle
(Croissant+CDIF+DID+ODRL)
│   ├── polaris_osi_plugin.py       Polaris extension scaffold (Python
pseudocode of the Java SPI)
│   ├── openlineage_emitter.py      emit RunEvent with OSIMetricFacet
│   └ ods_packager.py              wrap a Navigator bundle into an ODS data
product manifest
├ pyproject.toml
└ Makefile                        `make demo` runs the full pipeline end-to-
end
```

The code is a **reference implementation**, not a production library: the goal is to show that the data model in §5 round-trips, that the OSI → Croissant → CDIF → DID → ODRL projection is mechanical, and that Polaris can host all of it without redefining anything that Iceberg, OL, or OSI already define.

Sources

- Open Semantic Interchange specifications finalized — Snowflake
- OSI repository — [open-semantic-interchange/OSI](#)
- OSI working-group expansion — Snowflake
- OSI launch announcement (Snowflake / Salesforce / dbt Labs / BlackRock)
- Apache Polaris issue #4522 — Add OSI support
- OpenLineage object model docs

- [Apache Gravitino overview](#)
- [IPA Open Data Spaces initiative](#)
- [QueryGraph.ai](#)
- [QueryGraph reference implementation — querygraph/qg-python](#)
- [QueryGraph Rust implementation — querygraph/qg-rust](#)
- [Databricks blog — Semantic Layer Architecture](#)
- [Atlan — What is a Semantic Layer?](#)